CS 245, Spring 2026
Notes on Computability

Aditya Menon

Contents

Lecture 1: Turing Machines 2
1.1 Turing Machine 2
1.1.1 Tape Diagram 2
1.1.2 Behavior of the Machine 2
1.2 Configuration 2
1.3 Halting and Output 3
1.4 Transition diagram 4
Lecture 2: Turing Machines and Computability 6
2.1 Review of Turing Machines 6
2.2 Halting Problem 6
2.3 Turing Machines Doing Logic 7
2.4 Using Logic to Talk About Computation 7
Lecture 3: Using First-order logic to talk about Turing machines 8
3.1 Review 8
3.2 Sequences of Natural Numbers in First-order logic 8
3.3 Godel's incompleteness theorems 9

Lecture 1: Turing Machines

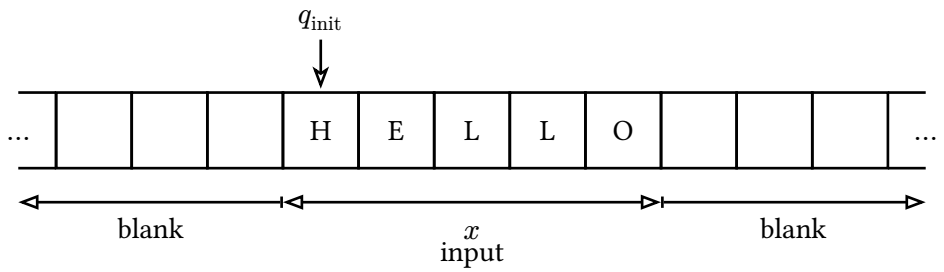
1.1 Turing Machine

Definition 1.1. A **Turing Machine** M is defined as

$$M = (Q, \Sigma, \delta, q_{\text{init}}, q_{\text{halt}})$$

where:
- Q: states
- Σ: alphabet (symbols)
- δ: transition function
- q_{init}: initial state
- q_{halt}: halt state

1.1.1 Tape Diagram



1.1.2 Behavior of the Machine

The behavior of the machine is governed by δ.

If the state is q and the tape cell we're looking at has symbol c, apply δ:

$$\delta(q, c) = (q', c', D), \quad D \in \{L, R, S\}$$

- Change state from q to q'.
- Overwrite c with c' on tape.
- Move left, right, or stay in place depending on D.

1.2 Configuration

A configuration will be a mathematical representation of a snapshot in the middle of the run: the contents of the tape, the internal state, and the position on the tape.

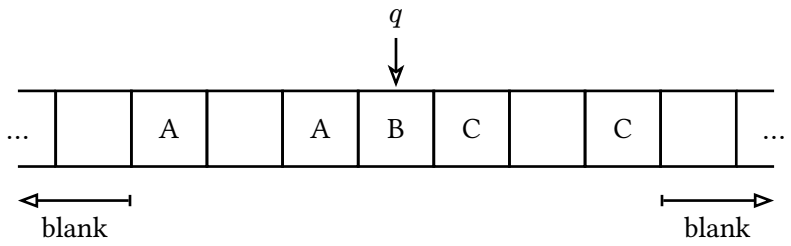
Definition 1.2. A configuration is:

$$s(q, c)t$$

where:

- s is a string over alphabet Σ whose first symbol is not the blank symbol.
- t is a string over Σ whose last symbol is not blank.
- $C \in \Sigma$.
- $q \in Q$.

Example 1.3.



The current configuration is:

$$\text{A\_A}(q, \text{B})\text{C\_C}$$

Definition 1.4. If we know the Turing machine M, a configuration uniquely defines the next configuration. We say the current configuration **yields** the next.

$$s(q, c)t \text{ yields}_M s'(q', c')t'$$

If $\delta(q, c) = (p, d, L)$ then:

- $q' = p$
- $t' = dt$ unless t is empty and $d =$ _ (blank), in which case t' is empty (so that last symbol of t' is not blank).
- c' is the last symbol of s, or blank if s is empty.
- s' is s with last symbol deleted, or empty if s is empty.

That defines the yields_M operation on configurations if δ said "L". We need a similar definition for R and S (we skip them).

If x is a string of non-blank symbols of M, then the **initial configuration** of M on x is $(q_{\text{init}}, \_)x$ (s is empty).

1.3 Halting and Output

M runs until it reaches q_{halt}, then stops. At that point, if the configuration is $s(q_{\text{halt}}, c)t$, the output is sct (tape minus infinite blanks to the left and right).

Definition 1.5. More formally, we say M **outputs** y **on input** x if there is a finite sequence of configurations:

$$C_1, C_2, ..., C_n$$

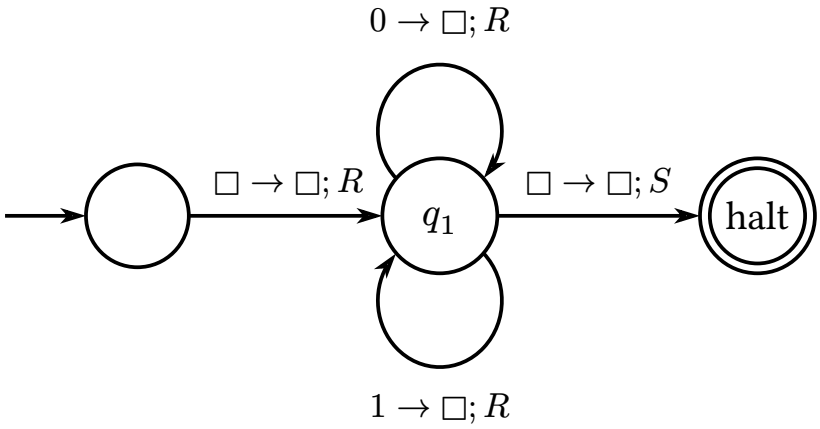
such that:

- C_1 is the initial config for M on input x,
- each C_i yields_M C_{i+1} for $i = 1, 2, ..., n-1$,
- the state in $C_1, C_2, ..., C_{n-1}$ is not q_{halt},
- the state in C_n is q_{halt},
- and $y = sct$ where $C_n = s(q_{\text{halt}}, c)t$.

On input x, M might either:

- **Output** a string y (uniquely defined), or
- **Never reach** q_{halt} (we say M "loops" on x)

1.4 Transition diagram



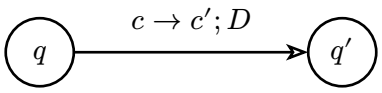
Where:

- Circles are states $q \in Q$.
- The halt state is double circled.
- The initial state has an arrow from nowhere.

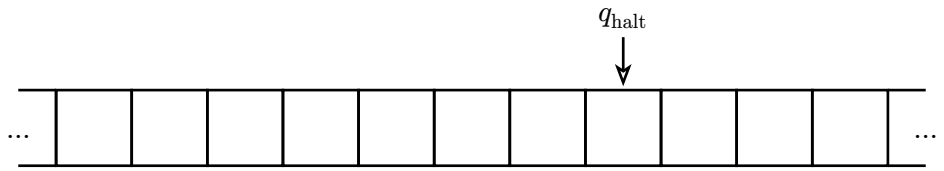
Each transition rule

$$\delta(q, c) = (q', c', D)$$

is an arrow:



Let's run the 3 state Turing machine on input 101.



$$(q_{\text{halt}}, \square)$$

so output is sct which is $\square$ (or really the empty string).

We can generally represent finite mathematical objects using strings over a fixed alphabet like $\{0, 1\}$. Eg:

- Integers like -15 (binary plus one bit for sign)
- Rational numbers (pairs of integers)
- Pairs of strings (represent as one string)
- Turing machines (they are finite!)

We use angle brackets for the string version of an object. Eg if M is a Turing machine,

$$\langle M \rangle$$

is a string representing M.

Question. What cannot be represented as a string? Arbitrary real numbers like

$$7.6362257\ldots$$

cannot be written as strings.

We can give Turing machines a finite mathematical object as input by converting it into a string.

For example, I can give as input

$$\langle M, x \rangle$$

where M is a Turing machine and x is a string, as input to a different Turing machine, say U.

Lecture 2: Turing Machines and Computability

2.1 Review of Turing Machines

A Turing machine is a mathematical object representing computation (a program).

A Turing machine M takes a string x as input, and either halts, returning a string y, or loops (never halts).

Recall: Angle brackets denote the string representation of a finite mathematical object.

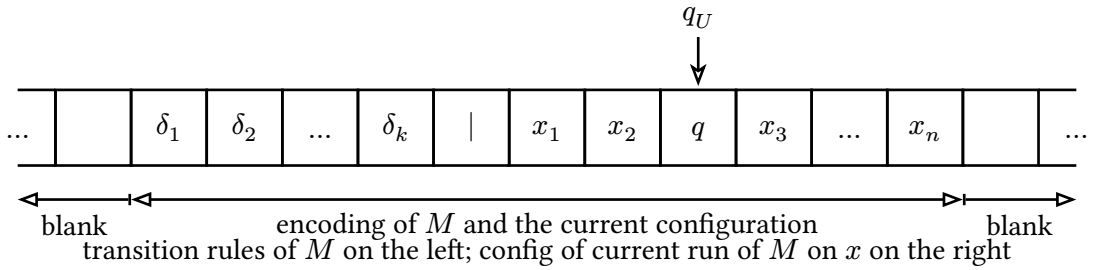
For example, $\langle M \rangle$ is a string if M is a Turing Machine.

Claim (Church-Turing Thesis): All reasonable ways of mathematically defining computation are **equivalent**, in that they can simulate each other. They are all also equivalent to real-life computer programs.

Theorem 2.1. *There is a Turing machine U (called a universal Turing machine) such that on input $\langle M, x \rangle$, $U(\langle M, x \rangle)$ runs $M(x)$:*

- *If $M(x)$ halts and returns y, then $U(\langle M, x \rangle)$ halts and returns y.*
- *If $M(x)$ loops, then $U(\langle M, x \rangle)$ loops.*

Idea: The tape of U looks like:



The head of U walks back and forth, possibly many times per step of M.

2.2 Halting Problem

Can we take an input $\langle M, x \rangle$ and output "halts" if $M(x)$ halts, "loops" if $M(x)$ loops, but always halt ourselves?

Definition 2.2 (Decidability). A set of strings is called a **language**.

A language L is called **decidable** if there is a Turing machine M such that:

- $M(x)$ outputs "yes" if $x \in L$.
- $M(x)$ outputs "no" if $x \notin L$.
- $M(x)$ **always** halts.

Question. Is the language

$$H = \{\langle M, x \rangle : M(x) \text{ halts}\}$$

decidable?

This question is equivalent to the halting problem.

Theorem 2.3. *H is not decidable.*

Proof. Suppose we had a program (i.e. a Turing machine) P which decides H. That is, $P(\langle M, x\rangle)$ tells you whether $M(x)$ halts.

Define a new program (Turing machine) Q as follows.

On input $\langle A\rangle$ (where A is a Turing machine), $Q(\langle A\rangle)$:

- First runs $P(\langle A, \langle A\rangle\rangle)$ to ask P whether $A(\langle A\rangle)$ halts.
- If P says $A(\langle A\rangle)$ halts, Q enters an infinite loop.
- If P says $A(\langle A\rangle)$ loops, Q halts.

What happens when we run $Q(\langle Q\rangle)$?

- Q runs $P(\langle Q, \langle Q\rangle\rangle)$ to ask P whether $Q(\langle Q\rangle)$ halts.
- If P says this halts, we enter a loop.
- If P says this loops, we halt.

Therefore, P is wrong, which is a contradiction. ■

2.3 Turing Machines Doing Logic

- The language of all formulas $L(\sigma)$ over a fixed signature σ is **decidable**. The same is true for the language of sentences over σ.
- If a set of sentences Γ over σ is decidable, then the set

$$\{\langle P, \phi\rangle : P \text{ is a formal proof of } \Gamma \vdash \phi\}$$

is decidable.

2.4 Using Logic to Talk About Computation

Fix the signature $\sigma = \{0, S, +, \times\}$ of Peano Arithmetic.

- We can represent **strings over $\{0, 1\}$** using **natural numbers** (e.g. in binary).
- We can do **string manipulation**, such as concatenating two strings, using their number encodings and the operations 0, S, $+$, and $\times$.
- For a Turing machine M, we can write a formula

$$\text{config}_M(x)$$

  which asserts the natural number x is a configuration of M (i.e., $s(q, c)t$).

- We can also write a formula

$$\text{yields}_M(x, y)$$

  which asserts x and y represent configurations, and that running M for one step from configuration x yields configuration y.

Lecture 3: Using First-order logic to talk about Turing machines

3.1 Review

Fix the signature

$$\sigma = \{0, S, +, \times\}.$$

- We can represent **strings** by natural numbers.
- We can perform string manipulation, such as concatenation.
- For a Turing machine M, we can construct a formula

$$\text{Config}_M(x)$$

which says that the string represented by x is formatted like a configuration of M, i.e., of the form

$$\langle s(q, c)t \rangle.$$

There is also a formula

$$\text{yields}_M(x, y)$$

which says that x and y represent configurations of M, and that running M for one step from the configuration represented by x produces the configuration represented by y.

We want a sentence

$$\phi_{M,x}$$

over σ which says that $M(x)$ halts. In other words, we want

$$\text{Val}_{\mathbb{N}}(\phi_{M,x}) = T \iff M(x) \text{ halts.}$$

Recall: saying that $M(x)$ halts means that there is a finite sequence of configurations

$$C_1, C_2, ..., C_n$$

such that:

- C_1 is the initial configuration of M on input x;
- C_n is in the halt state;
- each C_i is **not** in the halt state for $i < n$; and
- C_i yields$_M$ C_{i+1} for each $i < n$.

3.2 Sequences of Natural Numbers in First-order logic

Problem: How do we say

"There exists a finite sequence of natural numbers"

using first-order logic over

$$\sigma = \{0, S, +, \times\}?$$

Trick: Use prime factorization.

Consider

$$N = 2^{a_1} \times 3^{a_2} \times 5^{a_3} \times ... \times p_k^{a_k}.$$

We can represent a (finite) sequence of positive natural numbers

$$a_1, a_2, ..., a_k$$

by the **single** natural number

$$2^{a_1} \times 3^{a_2} \times ... \times p_k^{a_k}.$$

Thus, the statement

"There exists a finite sequence of configurations"

can be expressed as

"There exists a natural number \$N\$ whose prime-factorization exponents are natural numbers representing those configurations"

This leads to a valid sentence $\phi_{M,x}$ which is true in $\mathbb{N}$ if and only if $M(x)$ halts.

Furthermore, the function from strings to strings given by

$$\langle M, x \rangle \mapsto \langle \phi_{M,x} \rangle$$

is **computable**, meaning there is a Turing machine Encode such that

$$\text{Encode}(\langle M, x \rangle)$$

outputs

$$\langle \phi_{M,x} \rangle,$$

and Encode always halts.

Theorem 3.1. *There is no program (Turing machine) which takes as input a sentence over σ and determines whether it is true in $\mathbb{N}$.*

Equivalently, the language

$$\{\langle \phi \rangle : \phi \text{ is a sentence over } \sigma \text{ and } \text{Val}_{\mathbb{N}}(\phi) = T\}$$

is undecidable.

Proof. Suppose that such a program P existed.

Then given some Turing machine M and some input x,

$$P(\text{Encode}(\langle M, x \rangle))$$

would decide the halting problem. $\perp$ ■

3.3 Godel's incompleteness theorems

Theorem 3.2 (Gödel's first incompleteness theorem, weak version). *For any computer-checkable proof system which is **sound for** $\mathbb{N}$, there is a sentence ϕ over σ such that neither ϕ nor $\neg\phi$ can be proved in the system.*

Here, soundness for $\mathbb{N}$ means that whenever the system proves a sentence ϕ over σ, we have

$$\text{Val}_{\mathbb{N}}(\phi) = T.$$

*Such a proof system is therefore **incomplete**.*

Proof. Suppose, for contradiction, that there were a computer-checkable proof system which was both sound and complete for $\mathbb{N}$.

Define a Turing machine P as follows.

On input $\langle \phi \rangle$:

- Iterate over **all strings** s, first in order of length and then alphabetically:

$$\text{"", "0", "1", "00", "01", ...}$$

- For each string s:
  - check whether s is a proof of ϕ;
  - check whether s is a proof of $\neg\phi$.

If P finds a string s which proves ϕ, then P outputs

$$\text{true}$$

and halts.

If P finds a string s which proves $\neg\phi$, then P outputs

$$\text{false}$$

and halts.

Thus, if either ϕ or $\neg\phi$ has a proof in the proof system, then $P(\langle \phi \rangle)$ halts and outputs

$$\text{Val}_{\mathbb{N}}(\phi).$$

If every sentence ϕ has either a proof of ϕ or a proof of $\neg\phi$, then P **always** halts and correctly decides whether ϕ is true in $\mathbb{N}$.

But truth in $\mathbb{N}$ is undecidable. $\perp$ ■

Definition 3.3 (TM-expressive). A set of axioms Γ over σ is called **Turing-machine expressive**, or **TM-expressive**, if

$$\Gamma \vdash \underbrace{\text{Encode}(\langle M, x \rangle)}_{\phi_{M,x}} \quad \leftrightarrow \quad M(x) \ \text{ halts.}$$

Claim 1: Γ_{PA} is TM-expressive.

Proof sketch. Suppose first that

$$\Gamma_{\text{PA}} \vdash \phi_{M,x}.$$

Since PA is sound and $\mathbb{N}$ is a model of PA, it follows that

$$\text{Val}_{\mathbb{N}}(\phi_{M,x}) = T$$

and hence, $M(x)$ halts.

Conversely, suppose that $M(x)$ halts.

Then there is a finite sequence of configurations beginning with the initial configuration of M on input x and ending in a halting configuration.

PA can list all configurations in this finite sequence and prove, one by one, that every transition is correct. Since the sequence is finite, this produces a finite PA proof of $\phi_{M,x}$. ■

Theorem 3.4 (Gödel's first incompleteness theorem, strong version). *Let Γ be a set of axioms over σ.*

If Γ is

- *decidable,*
- *TM-expressive, and*
- *consistent,*

then there is some sentence ϕ over σ such that

$$\Gamma \nvdash \phi \text{ and } \Gamma \nvdash \neg\phi.$$

i.e., Γ is incomplete for $\mathbb{N}$.

Theorem 3.5 (Gödel's second incompleteness theorem). *Let Γ be a decidable set of axioms over σ.*

We can construct a sentence ϕ_Γ which says that Γ is consistent.

More precisely, ϕ_Γ says that there is no proof of

$$\Gamma \vdash \bot.$$

Equivalently, it says that a Turing machine which searches for such a proof and halts when it finds one will never halt.

Then, If Γ is TM-expressive and consistent,

$$\Gamma \nvdash \phi_\Gamma.$$

Remark 3.6 (Self Reference). Consider the expression

"This sentence is false."

We initially wanted to say that it is not a sentence at all.

- We might try to ban talking about sentences, but this fails because natural numbers can encode and talk about strings and sentences.
- We might try to ban self-reference, but Turing machines are great at self-reference, and logic and arithmetic can talk about Turing machines.